

Types of computers:

↳ There are two types of computers

Fixed Program

- ↳ Performs only a specific task
- ↳ Cannot be re-programmed to do other tasks

Stored program

- ↳ Performs different tasks
- ↳ Can be reprogrammed

Von Nuemann Architecture

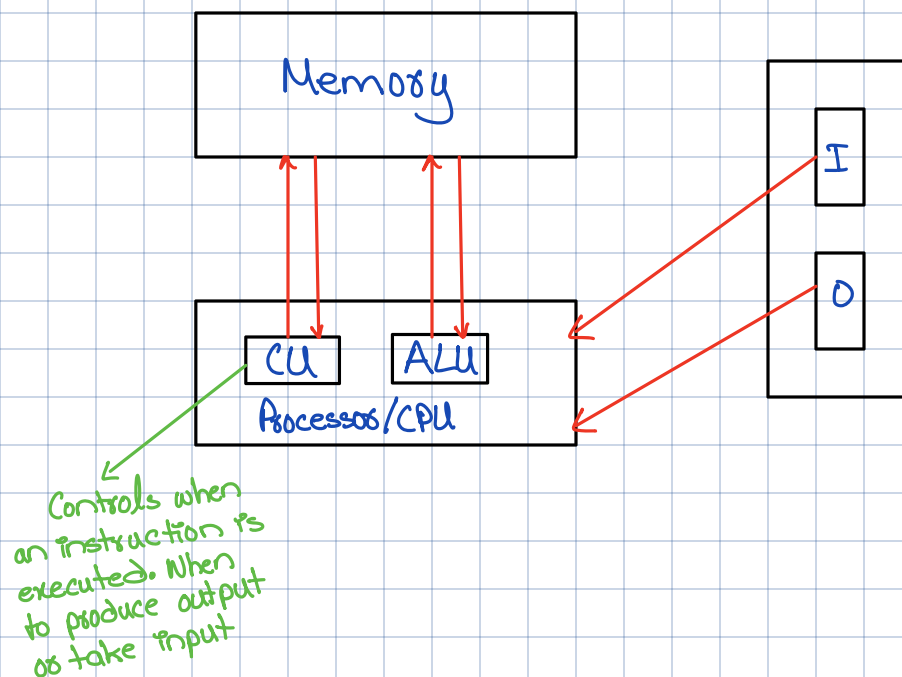
↳ Based on three basic characteristics:

1. It has four basic components:

1. Memory
 2. ALU
 3. Control Unit
 4. Input/Output System (I/O)
- } combined into CPU

2. Program is stored in memory during execution

3. Program instructions are executed sequentially



Memory subsystem:

- ↳ Consists of memory cells of fixed size.
- ↳ Each cell has a address associated with it.

Words:

- ↳ A Basic unit that is stored/retrieved from RAM
- ↳ Can be of varying size.
- ↳ Usually 8 bits or 1 byte = 1 word

↳ RAM size is multiples of 8

↳ N is the number of address bits, 2^N is total number of addressable cells.

Example,

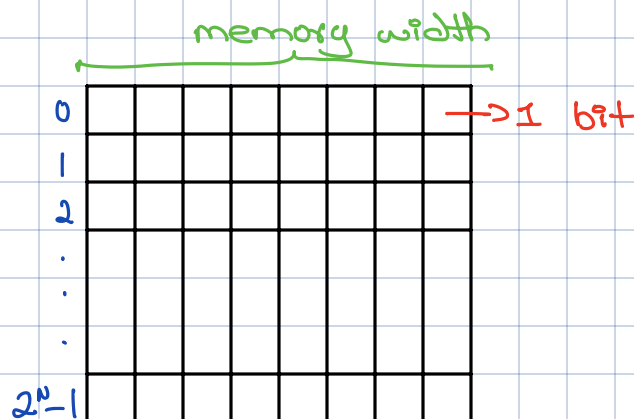
$$N=3, 2^3=8$$

0 0 0
0 0 1
0 1 0
0 1 1
1 0 0
1 0 1
1 1 0
1 1 1

8 cells with 3
bit unique address

Also called address space

↳ Control Unit tells which address corresponds to which cell.



Operations on memory

1. Fetch/Read

- ↳ Copy contents of a memory cell with specified address

2. Write/Store

- ↳ Store specified value into memory cell specified by address

Input/Output Subsystem:

- ↳ Handles device that allow the computer system to:
 - ↳ Communicate and interact with the outside world
 - ↳ Store information (HDDs, CD-ROMs)

The Problem: Speed Difference Between RAM and I/O Devices

RAM (Random Access Memory):

- Very fast. It can access data in around 50 nanoseconds.
- Hard Drive (I/O Device) is much slower. It takes around 10 milliseconds to access data, which is 10,000,000 nanoseconds.
- This huge difference in speed creates a problem: if the processor has to wait for the hard drive to read or write data, it will waste a lot of time being idle.

The Solution: I/O Controller

- An I/O controller is a special-purpose processor that acts as a middleman between the CPU (main processor), RAM, and the I/O device (e.g., hard drive).
- What the I/O Controller Does:
 - **Buffering Data:**
 - The I/O controller uses a memory buffer to temporarily store data being transferred between the hard drive (or any other I/O device) and RAM.
 - This allows the slow I/O device to transfer data at its own pace without directly holding up the fast CPU.
 - **Processor Offloading:**
 - While the I/O controller handles the data transfer, the CPU is free to perform other tasks instead of waiting for the slow I/O operation to complete.

How It Works in Practice:

1. The CPU requests some data from the hard drive (e.g., a file to process).
2. Instead of waiting for the hard drive to slowly send the data, the CPU tells the I/O controller to handle the request.
3. The I/O controller fetches the requested data from the hard drive and temporarily stores it in a buffer (a small area in memory).
4. While this is happening, the CPU works on other tasks (e.g., computations, running programs, etc.).
5. Once the data is ready in the buffer, the CPU accesses it from the RAM or the buffer directly, which is much faster

ALU Subsystem:

- ↳ Performs mathematical & logic operations
- ↳ Integrated into CPU

↳ Consist of:

- ↳ Circuits to do the arithmetic/logic operations
- ↳ Registers to store intermediate results

Control Unit:

↳ Execute program repeatedly

1. Fetch from memory the next instruction to be executed
2. Decode it, that is, determine what is to be done
3. Execute it by issuing the appropriate signals to the ALU, memory and I/O subsystems
4. Continues until HALT (end) instruction

Bus:

↳ Communication line that is used to transfer from one part of computer to another.

1. Address bus → Communicates address to diff components
2. Data bus → Communicates data to diff components
3. Control bus → Sends control signals

Machine level instructions:

op code Address 1 Address 2
≤ 8 bits

Program Counter

- **Purpose:** The Program Counter keeps track of where the CPU is in the execution of a program.
- **Contents:** It stores the address of the instruction in memory that the CPU needs to execute next.

How Does It Work?

• At the Start of Execution:

- The PC is initialized to the memory address of the first instruction of the program.

- **During Execution:**

- The CPU reads the instruction at the address pointed to by the PC.
- After fetching the instruction, the PC is automatically updated to point to the address of the next instruction in memory.